Top-K & Heaps

> → **Key Idea**: Use heaps to efficiently maintain top-k elements or find extremes in streaming data

> • **When to use**: Finding top-k elements, streaming data, priority scheduling, median finding

> ⊕ **Complexity**: Time O(n log k) for top-k, Space O(k)

> ⚠ **Common Pitfall**: Using wrong heap type (min vs max) or not maintaining heap size

Overview

Visual Example

Min-Heap Operations Visualization

```
Min-Heap Property: Parent ≤ Children

        [2] (Root - Smallest)
       /   \
     [5]    [12]
    /   \   /
  [17] [8] [20]

Array Representation: [2, 5, 12, 17, 8, 20]
Indices:               0  1   2   3  4   5

Left Child = 2*i + 1
Right Child = 2*i + 2
Parent = (i - 1) / 2
```

Core Implementation

Basic Heap Implementation

```java
import java.util.*;

public class MinHeap {
    private List<Integer> heap;

    public MinHeap() {
```

```
        heap = new ArrayList<>();
    }

    public void insert(int val) {
        heap.add(val);
        heapifyUp(heap.size() - 1);
    }

    public int extractMin() {
        if (heap.isEmpty()) {
            throw new IllegalStateException("Heap is empty");
        }

        int min = heap.get(0);
        int lastVal = heap.remove(heap.size() - 1);

        if (!heap.isEmpty()) {
            heap.set(0, lastVal);
            heapifyDown(0);
        }

        return min;
    }

    public int peek() {
        if (heap.isEmpty()) {
            throw new IllegalStateException("Heap is empty");
        }
        return heap.get(0);
    }

    private void heapifyUp(int index) {
        while (index > 0) {
            int parentIndex = (index - 1) / 2;

            if (heap.get(index) >= heap.get(parentIndex)) {
                break;
            }

            swap(index, parentIndex);
            index = parentIndex;
        }
    }

    private void heapifyDown(int index) {
        while (true) {
            int smallest = index;
            int leftChild = 2 * index + 1;
            int rightChild = 2 * index + 2;

            if (leftChild < heap.size() &&
                heap.get(leftChild) < heap.get(smallest)) {
                smallest = leftChild;
            }

            if (rightChild < heap.size() &&
```

```
                heap.get(rightChild) < heap.get(smallest)) {
                smallest = rightChild;
            }

            if (smallest == index) {
                break;
            }

            swap(index, smallest);
            index = smallest;
        }
    }

    private void swap(int i, int j) {
        int temp = heap.get(i);
        heap.set(i, heap.get(j));
        heap.set(j, temp);
    }

    public int size() {
        return heap.size();
    }

    public boolean isEmpty() {
        return heap.isEmpty();
    }
}
```

Priority Queue Using Java Collections

```
import java.util.PriorityQueue;
import java.util.Collections;

public class HeapExamples {

    // Min-heap (default in Java)
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();

    // Max-heap (reverse order)
    PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder())

    // Custom comparator
    PriorityQueue<int[]> customHeap = new PriorityQueue<>((a, b) -> a[1] - b[1]);

    // String priority queue (lexicographic order)
    PriorityQueue<String> stringHeap = new PriorityQueue<>();
}
```

Key Optimizations

1. Top-K Elements Optimization

```
public List<Integer> topKElements(int[] nums, int k) {
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();

    for (int num : nums) {
        minHeap.offer(num);

        if (minHeap.size() > k) {
            minHeap.poll(); // Remove smallest
        }
    }

    return new ArrayList<>(minHeap);
}

// Alternative: Use selection algorithm for O(n) average case
public int quickSelect(int[] nums, int k) {
    return quickSelectHelper(nums, 0, nums.length - 1, nums.length - k);
}

private int quickSelectHelper(int[] nums, int left, int right, int targetIndex) {
    if (left == right) return nums[left];

    int pivotIndex = partition(nums, left, right);

    if (pivotIndex == targetIndex) {
        return nums[pivotIndex];
    } else if (pivotIndex < targetIndex) {
        return quickSelectHelper(nums, pivotIndex + 1, right, targetIndex);
    } else {
        return quickSelectHelper(nums, left, pivotIndex - 1, targetIndex);
    }
}
```

2. Merge K Sorted Lists Optimization

```
// Optimized approach using divide and conquer
public ListNode mergeKLists(ListNode[] lists) {
    if (lists == null || lists.length == 0) return null;

    while (lists.length > 1) {
        List<ListNode> mergedLists = new ArrayList<>();

        for (int i = 0; i < lists.length; i += 2) {
            ListNode l1 = lists[i];
            ListNode l2 = (i + 1 < lists.length) ? lists[i + 1] : null;
            mergedLists.add(merge2Lists(l1, l2));
        }
```

```
            lists = mergedLists.toArray(new ListNode[0]);
        }

        return lists[0];
    }

    private ListNode merge2Lists(ListNode l1, ListNode l2) {
        ListNode dummy = new ListNode(0);
        ListNode current = dummy;

        while (l1 != null && l2 != null) {
            if (l1.val <= l2.val) {
                current.next = l1;
                l1 = l1.next;
            } else {
                current.next = l2;
                l2 = l2.next;
            }
            current = current.next;
        }

        current.next = (l1 != null) ? l1 : l2;
        return dummy.next;
    }
```

Common Problem Patterns

1. Top-K Frequent Elements

Problem: Given an integer array nums and an integer k, return the k most frequent elements.

Example:

```
Input: nums = [1,1,1,2,2,3], k = 2
Output: [1,2]
```

```
    public int[] topKFrequent(int[] nums, int k) {
        // Count frequency
        Map<Integer, Integer> frequencyMap = new HashMap<>();
        for (int num : nums) {
            frequencyMap.put(num, frequencyMap.getOrDefault(num, 0) + 1);
        }

        // Use min-heap to track top k frequent elements
        PriorityQueue<int[]> minHeap = new PriorityQueue<>((a, b) -> a[1] - b[1]);

        for (Map.Entry<Integer, Integer> entry : frequencyMap.entrySet()) {
            minHeap.offer(new int[]{entry.getKey(), entry.getValue()});
```

```
            if (minHeap.size() > k) {
                minHeap.poll();
            }
        }

        // Extract results
        int[] result = new int[k];
        for (int i = k - 1; i >= 0; i--) {
            result[i] = minHeap.poll()[0];
        }

        return result;
    }
```

2. Kth Largest Element in Array

Problem: Find the kth largest element in an unsorted array.

Example:

```
Input: nums = [3,2,1,5,6,4], k = 2
Output: 5
```

```
    public int findKthLargest(int[] nums, int k) {
        PriorityQueue<Integer> minHeap = new PriorityQueue<>();

        for (int num : nums) {
            minHeap.offer(num);

            if (minHeap.size() > k) {
                minHeap.poll();
            }
        }

        return minHeap.peek();
    }

    // Alternative: Quick Select approach
    public int findKthLargestQuickSelect(int[] nums, int k) {
        return quickSelect(nums, 0, nums.length - 1, nums.length - k);
    }

    private int quickSelect(int[] nums, int left, int right, int targetIndex) {
        int pivotIndex = partition(nums, left, right);

        if (pivotIndex == targetIndex) {
            return nums[pivotIndex];
        } else if (pivotIndex < targetIndex) {
            return quickSelect(nums, pivotIndex + 1, right, targetIndex);
        } else {
```

```
            return quickSelect(nums, left, pivotIndex - 1, targetIndex);
        }
    }

    private int partition(int[] nums, int left, int right) {
        int pivot = nums[right];
        int i = left;

        for (int j = left; j < right; j++) {
            if (nums[j] <= pivot) {
                swap(nums, i, j);
                i++;
            }
        }

        swap(nums, i, right);
        return i;
    }
```

3. Find Median from Data Stream

Problem: Design a data structure that supports adding numbers and finding median efficiently.

```
class MedianFinder {
    private PriorityQueue<Integer> maxHeap; // smaller half
    private PriorityQueue<Integer> minHeap; // larger half

    public MedianFinder() {
        maxHeap = new PriorityQueue<>(Collections.reverseOrder());
        minHeap = new PriorityQueue<>();
    }

    public void addNum(int num) {
        // Add to maxHeap first
        maxHeap.offer(num);

        // Ensure maxHeap.top <= minHeap.top
        minHeap.offer(maxHeap.poll());

        // Balance heaps (maxHeap can have at most 1 extra element)
        if (minHeap.size() > maxHeap.size()) {
            maxHeap.offer(minHeap.poll());
        }
    }

    public double findMedian() {
        if (maxHeap.size() > minHeap.size()) {
            return maxHeap.peek();
        } else {
            return (maxHeap.peek() + minHeap.peek()) / 2.0;
        }
```

```
        }
    }
```

4. Merge K Sorted Lists

Problem: Merge k sorted linked lists and return it as one sorted list.

Example:

```
Input: lists = [[1,4,5],[1,3,4],[2,6]]
Output: [1,1,2,3,4,4,5,6]
```

```java
public ListNode mergeKLists(ListNode[] lists) {
    if (lists == null || lists.length == 0) return null;

    PriorityQueue<ListNode> minHeap = new PriorityQueue<>((a, b) -> a.val - b.val);

    // Add first node of each list to heap
    for (ListNode head : lists) {
        if (head != null) {
            minHeap.offer(head);
        }
    }

    ListNode dummy = new ListNode(0);
    ListNode current = dummy;

    while (!minHeap.isEmpty()) {
        ListNode smallest = minHeap.poll();
        current.next = smallest;
        current = current.next;

        if (smallest.next != null) {
            minHeap.offer(smallest.next);
        }
    }

    return dummy.next;
}
```

5. Task Scheduler

Problem: Given tasks and cooldown time, find minimum intervals needed.

Example:

```
Input: tasks = ["A","A","A","B","B","B"], n = 2
Output: 8
Explanation: A -> B -> idle -> A -> B -> idle -> A -> B
```

```java
public int leastInterval(char[] tasks, int n) {
    // Count task frequencies
    int[] frequencies = new int[26];
    for (char task : tasks) {
        frequencies[task - 'A']++;
    }

    // Use max-heap to track most frequent tasks
    PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder())
    for (int freq : frequencies) {
        if (freq > 0) {
            maxHeap.offer(freq);
        }
    }

    int time = 0;
    Queue<int[]> cooldown = new LinkedList<>(); // [frequency, available_time]

    while (!maxHeap.isEmpty() || !cooldown.isEmpty()) {
        time++;

        // Add tasks back from cooldown
        if (!cooldown.isEmpty() && cooldown.peek()[1] == time) {
            maxHeap.offer(cooldown.poll()[0]);
        }

        // Execute most frequent task
        if (!maxHeap.isEmpty()) {
            int freq = maxHeap.poll() - 1;
            if (freq > 0) {
                cooldown.offer(new int[]{freq, time + n + 1});
            }
        }
    }

    return time;
}
```

Advanced Variations

Sliding Window Maximum

```java
public int[] maxSlidingWindow(int[] nums, int k) {
    if (nums == null || nums.length == 0) return new int[0];

    PriorityQueue<int[]> maxHeap = new PriorityQueue<>((a, b) -> b[0] - a[0]);
    int[] result = new int[nums.length - k + 1];

    for (int i = 0; i < nums.length; i++) {
        maxHeap.offer(new int[]{nums[i], i});

        // Remove elements outside window
        while (!maxHeap.isEmpty() && maxHeap.peek()[1] <= i - k) {
            maxHeap.poll();
        }

        // Record maximum for current window
        if (i >= k - 1) {
            result[i - k + 1] = maxHeap.peek()[0];
        }
    }

    return result;
}
```

K Closest Points to Origin

```java
public int[][] kClosest(int[][] points, int k) {
    PriorityQueue<int[]> maxHeap = new PriorityQueue<>((a, b) ->
        (b[0] * b[0] + b[1] * b[1]) - (a[0] * a[0] + a[1] * a[1]));

    for (int[] point : points) {
        maxHeap.offer(point);

        if (maxHeap.size() > k) {
            maxHeap.poll();
        }
    }

    int[][] result = new int[k][2];
    for (int i = k - 1; i >= 0; i--) {
        result[i] = maxHeap.poll();
    }

    return result;
}
```

Meeting Rooms II

```java
public int minMeetingRooms(int[][] intervals) {
    if (intervals == null || intervals.length == 0) return 0;

    Arrays.sort(intervals, (a, b) -> a[0] - b[0]);

    PriorityQueue<Integer> minHeap = new PriorityQueue<>();

    for (int[] interval : intervals) {
        // Remove meetings that have ended
        if (!minHeap.isEmpty() && minHeap.peek() <= interval[0]) {
            minHeap.poll();
        }

        minHeap.offer(interval[1]);
    }

    return minHeap.size();
}
```

Interview Tips

Pattern Recognition

- **Top-K problems**: Use heap of size K to avoid sorting entire array
- **Streaming data**: Heaps maintain running maximum/minimum efficiently
- **Merge operations**: Priority queue orders elements from multiple sources
- **Scheduling problems**: Heaps track earliest available times/deadlines

Common Mistakes

- **Wrong heap type**: Using max-heap when min-heap is needed (or vice versa)
- **Heap size management**: Not maintaining fixed size for top-K problems
- **Comparator errors**: Incorrect custom comparison functions
- **Empty heap access**: Not checking if heap is empty before peek/poll

Implementation Variations

```java
// Custom object in heap
class Task {
    String name;
    int priority;

    Task(String name, int priority) {
        this.name = name;
```

```
            this.priority = priority;
        }
    }

    PriorityQueue<Task> taskQueue = new PriorityQueue<>((a, b) -> b.priority - a.priorit

    // Multiple criteria comparison
    PriorityQueue<int[]> complexHeap = new PriorityQueue<>((a, b) -> {
        if (a[0] != b[0]) return a[0] - b[0]; // First by first element
        return a[1] - b[1]; // Then by second element
    });

    // Lazy deletion for sliding window
    Map<Integer, Integer> toRemove = new HashMap<>();
    while (!maxHeap.isEmpty() && toRemove.containsKey(maxHeap.peek())) {
        int val = maxHeap.poll();
        toRemove.put(val, toRemove.get(val) - 1);
        if (toRemove.get(val) == 0) toRemove.remove(val);
    }
```

Time & Space Complexity

Operation	Time Complexity	Space Complexity	Notes
Insert	O(log n)	O(1)	Single heap operation
Extract Min/Max	O(log n)	O(1)	Single heap operation
Peek	O(1)	O(1)	No modification
Top-K Elements	O(n log k)	O(k)	k-sized heap
Merge K Lists	O(n log k)	O(k)	k nodes in heap
Find Median (stream)	O(log n)	O(n)	Two heaps
Quick Select	O(n) average, $O(n^2)$ worst	O(1)	In-place partition

← Previous: Intervals | Back to Main Guide | Next: Greedy Algorithms →